Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Callback

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Callback
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Callback

1 Callback

Study focus: Callback. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000014 · EV0000015

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

I am Nita Anne Jacob, Assistant Professor from the Department of Computer Science and Engineering.

Key details

- In this video, we're going to continue our discussion with functions, and what we will explore is the concept of callback.
- We know that you can pass any arguments to functions as input.
- But when a function is passed to another function as an argument, which is in turn invoked inside the outer function to complete a specific task, it is known as a function callback.
- Show you how exactly it works and what the concept really is.
- If I want to change this to F two, it's going to say.
- And the input to the interface is just one parameter which I'm just going to name as Fn the leader and internally and the body of the function, I'm going to just say fn and a function call.
- Whatever parameter or argument you pass here, it's going to be invoked as a function, which means the value of fn must be a function object.
- The function objects I have are F1 and F2.
- The functions F1 and F2 are called through the interface.
- Let me rephrase that.
- Fn is a function object that calls back the functions F1 or F2.
- I'm not going to change F1 or F2 here, but rather just pass the name of the function object.
- I'm going to clear the screen so you can see the error clearly.
- It says int object is not callable, which means in callback, the parameter.
- must receive a function object.

- A calling function can call the callback function as many times required, and all you have to do is just call the callback function once.

Example or application from the lecture

- So let me demonstrate this with an example to help you understand it better.
- And if I were to invoke F1, okay, let's go over to the interpreter, type Python.
- Let's look at the command prompt.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000014 · EV0000015 · EV0000016 · +1 source units in pack.json

Core explanation · 05:00–09:59

Within the interface function, I could have put a loop of say calling it four times, right?

Key details

- We're trying to just show the user what exactly the user needs to see.
- He doesn't need to know what are the intricacies of the application.
- Sorry, to destination.
- just say printing booked in Uber.
- I'm going to repeat the same task for Ola and create another function, Ola.
- If you open your Google Maps application and type a particular destination, apart from just the directions and how much time it takes from point A to point B, via car, bus, bike, walking, it also gives you an option to hire a cab.
- Why not exiting from Google and then going to Ola or Uber?
- When I invok app, it must get source and destination as the parameters as well.
- Google Maps function.
- If the value in app is equal to Uber.
- You can do that when you try this out.
- I if it's Uber, what I'm going to do is call G maps with source, destination, and app.
- But what will the app be?
- It's going to be a string, so I'm going to pass the function object Uber.
- I've taken care of Uber.
- I have to talk about Ola instead.

Example or application from the lecture

- One such example is that of your Google Maps.
- So I'm going to write remove this entire code and emulate an example of how we would use callback in Google Maps.
- And let's say I want to just book a cab from some source to a destination.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000018 · EV0000019 · EV0000020 · +1 source units in pack.json

Core explanation · 10:00–14:59

input errors like say else if and then again an else enter a correct app I'm just going to assume that my user is going to use the correct input right in.

Key details

- Ideal in an ideal world, this is an incorrect form of programming where I'm not taking care of the mistakes a user would make.
- But this is just to illustrate the concept of callbacks.
- It's not else if you have to give LF app equal to hola.
- And valid application.
- Space Electronic City.
- via Uber in uppercase letters, it's going to say enter valid application because we've only taken care of Uber and Ola in terms of case sensitive strings.
- I hope you have understood the concept of callback.
- App will be replaced with Uber and you're saying Uber of Source and Destination.
- I hope you've understood this concept.
- Do try this entire program out on Python Tutor to understand it further.
- I hope you will practice this and also try all the worksheets that are available on Pesso Academy.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000022 · EV0000023 · EV0000024 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Callback transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Callback transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Callback transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Callback transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00006	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00007	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00008	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00009	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

C Revision and retrieval practice